PRACTICE WALKTHROUGH · WEEK 2

Git and VS Code, Step by Step

Clone, edit, commit and push · about 15 minutes of setup once, then about 10 minutes

Goal: by the end you will have cloned a repository, created a file, committed it and pushed it to GitHub from VS Code, and you will have seen the result on github.com.

SET UP	STEP 1	STEP 2	STEP 3	STEP 4	CHECK
Install	**Connect**	**Clone**	**Create a file**	**Commit and push**	**On GitHub**
once	2 min	3 min	2 min	3 min	1 min

A GitHub account	Free. Sign up at github.com/signup if you do not have one.
Git	The tool that tracks changes. Step 0 shows how to check and install it.
VS Code	The editor you will use. Free from code.visualstudio.com.
Internet	Needed to sign in, clone and push.
The repository	`https://github.com/testing03-03/ecomMart.git`

Pick your path before Step 2

You can only push to a repository you own or have been added to as a collaborator. A quick rule decides which path you use. Not sure? Choose Path B. It always works.

PATH A

You own it, or you are a collaborator

Use this if you sign in to GitHub as `testing03-03`, or Mudit has added your account as a collaborator.

You clone the original repository and push straight to it.

You will see the **Settings** tab on the repository page if you own it.

PATH B

Anyone else (a personal copy)

You **fork** the repository first. A fork is your own copy under your account.

You clone your fork and push to it. Nothing you do can affect the original.

This is also the safest way to practise.

Step 0: One-time setup (about 15 minutes)

Skip any item you have already done. Each row ends with a check so you know it worked.

0 Install and prepare your tools

once

#	Do this	You should see
0.1	Check whether Git is already installed. Open a terminal. **Windows:** press Start, type `Command Prompt`, press Enter. **Mac:** press Cmd + Space, type `Terminal`, press Enter. Type `git --version` and press Enter.	A line such as `git version 2.x.x`. If you see it, skip to 0.3.
0.2	Install Git if it was missing. **Windows:** go to `git-scm.com/download/win`, run the installer, and accept the default options on every screen. **Mac:** if a pop-up offers to install developer tools, click **Install** and wait for it to finish. Close the terminal, open a new one, and repeat 0.1.	The `git version` line appears.

> **Stuck?** On Windows, `'git' is not recognized` means the terminal started before the install finished. Close every terminal and VS Code, open them again, and retry. If it still fails, restart the computer.

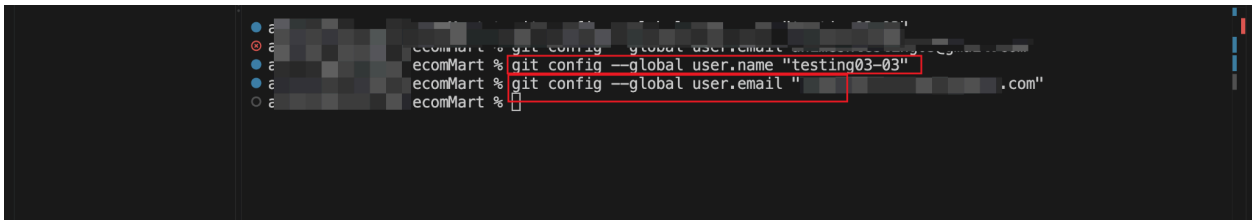
0.3	Install VS Code if you do not have it. Go to code.visualstudio.com, click **Download**, run the installer and accept the defaults. Open VS Code once.	A VS Code window opens with a Welcome tab.
0.4	Create a GitHub account if you do not have one: github.com/signup. Confirm the email GitHub sends you.	You can sign in at github.com and see your dashboard.

#	Do this	You should see

0.5	Tell Git who you are. This is needed once per computer, or your first commit will fail. In the terminal, type each line with your own details and press Enter after each.
	`git config --global user.name "Your Name"`
	`git config --global user.email "you@example.com"`
	Use the email on your GitHub account.
		No message appears. To check, type `git config --global user.name`. It prints your name.

Stuck? If you chose "Keep my email private" on GitHub, use the address shown at GitHub **Settings > Emails**. It ends with `users.noreply.github.com`.





1 Connect VS Code to GitHub

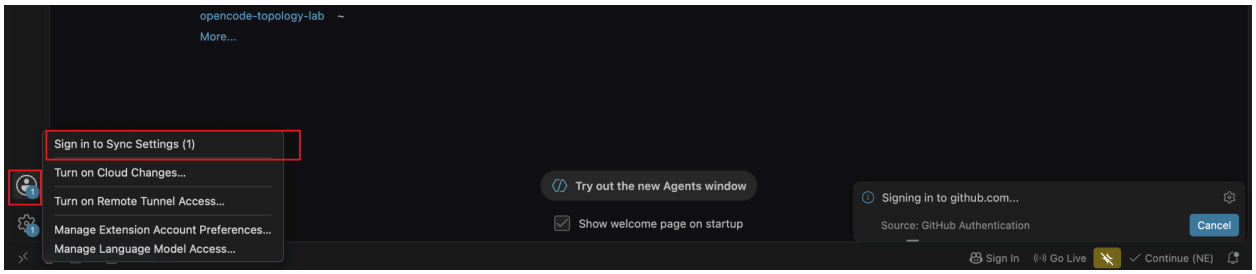
about 2 min

#	Do this	You should see
1.1	Open VS Code.	The editor window.
1.2	Find the **Accounts** icon at the bottom of the thin vertical bar on the left. It looks like the outline of a person. Click it.	A small menu opens with sign-in options.
1.3	Choose the option to **sign in with GitHub**. The exact wording changes between VS Code versions, so pick the one that mentions GitHub.	Your web browser opens on github.com and asks you to authorize **Visual Studio Code**.

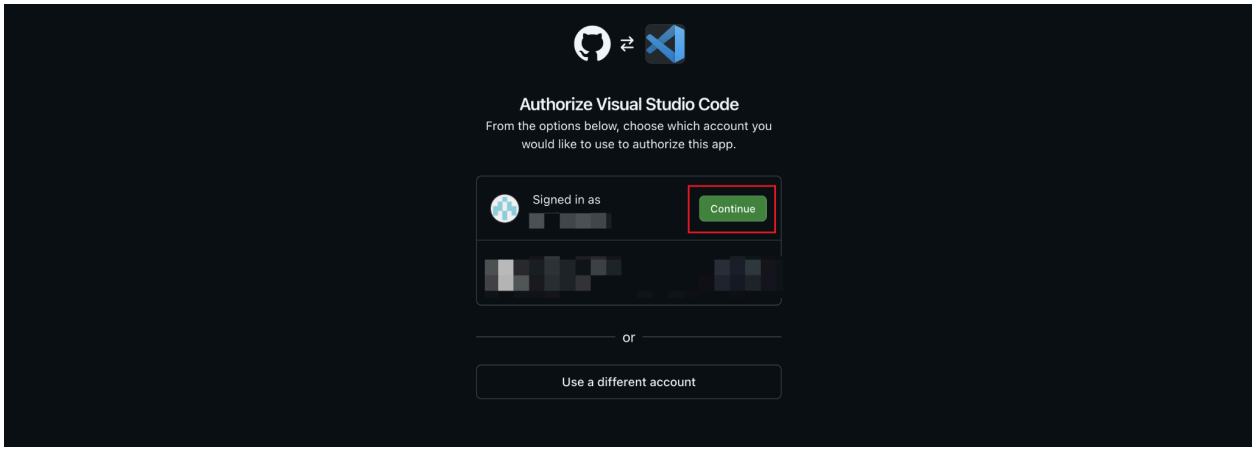
#	Do this	You should see
1.4	In the browser, click **Continue** or **Authorize**. If the browser asks whether to open VS Code, allow it.	You are back in VS Code. Click the Accounts icon again and your GitHub username is listed.
1.5	Confirm Git works inside VS Code. Click the **Terminal** menu, then **New Terminal**. Type `git --version` and press Enter.	A version number, such as `git version 2.x.x`.

> **Stuck?** If you skipped sign-in, nothing breaks yet. VS Code will ask you to sign in again when you clone or push. Do it then.

Step 1.2 -



Step 1.3 -



Step 1.5 -

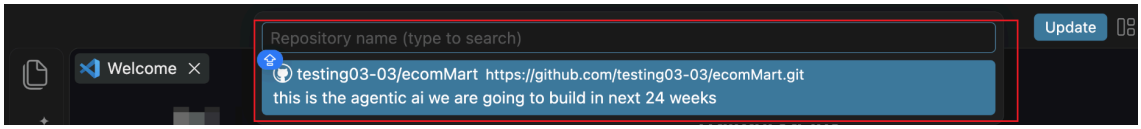


2 Clone the repository

about 3 min

#	Do this	You should see
2.1	Open the repository in your browser while signed in to GitHub. **Path A:** go to github.com/testing03-03/ecomMart.git **Path B:** go to the same address, click **Fork** (top right), then **Create fork**.	Path A: the repository page. Path B: you land on `github.com/YOUR-USERNAME/PostgreSQL_Ex01_UNESCO`, and "forked from testing03-03" appears under the name.
2.2	Click the green **Code** button. Choose the **Local** tab, then **HTTPS**. Click the copy icon next to the address.	An address starting with `https://github.com/` and ending in `.git`. Path A has `testing03-03` in it. Path B has your username.
2.3	Create a new empty folder on your computer for the project, for example `Documents/git-demo`.	An empty folder exists.
2.4	In VS Code press **Ctrl + Shift + P** (Windows) or **Cmd + Shift + P** (Mac). Type `Git: Clone` and press Enter. (Another way: open the Source Control icon and click **Clone Repository**.)	A box asking you to provide the repository URL.
2.5	Paste the address (Ctrl or Cmd + V) and press Enter. If a sign-in window appears, sign in with GitHub.	A folder picker opens.
2.6	Select the folder from 2.3 and confirm with the button at the bottom of the picker (usually **Select as Repository Destination**).	A "Cloning git repository" message appears in the bottom-right corner.
2.7	When VS Code asks "Would you like to open the cloned repository?", click **Open**. If it asks whether you trust the authors, click **Yes**.	The Explorer on the left shows `PostgreSQL_Ex01_UNESCO` containing `README.md`. The bottom-left corner shows `main`.

Stuck? "Destination path already exists": pick a new empty folder. **"Authentication failed":** sign in through the browser pop-up. GitHub no longer accepts your account password for Git, so do not type it. **Nothing happens after Enter:** check the address has no spaces and ends in `.git`.



3 Create or edit a file

about 2 min

#	Do this	You should see
3.1	In the Explorer, move your mouse over the project name. Click the **New File** icon (a page with a plus sign).	A text box appears for the file name.
3.2	Type `week2-demo.md` and press Enter.	An empty file opens in the editor.
3.3	Click in the editor and type these four lines:	Your text appears. The file tab shows a white dot, meaning it is not saved yet.

eComMART Agentic Commerce System — Build Approach

Squad: AI Engineering Academy — eComMART AI Squad
Programme: 24-Week Agentic AI Engineering Course
Status: Weeks 1–6 in progress

The Problem

eComMART is a multinational B2B marketplace. As the platform scales, two things are breaking down.

Buyers submit RFQs (Requests for Quotation) and wait days for a response because every supplier is replying manually. Suppliers spend most of their working day qualifying leads, checking stock levels, and answering the same questions repeatedly — work that follows predictable rules and should not require a human every time.

The business needs an AI system that handles this loop autonomously, safely, and within defined policy guardrails.

The Solution — Two Cooperating Agents

Seller Agent

A virtual storefront assistant for suppliers. It reads incoming buyer RFQs, checks live inventory, qualifies the lead using standard business frameworks, drafts a price quote, counters low offers within the supplier's floor price policy, and escalates anything outside policy to a human.

Buyer Agent

A procurement assistant for buyers. It matches a buyer's brief against supplier catalogues, surfaces the best options, submits RFQs on the buyer's behalf, and tracks negotiation progress.

Both agents share the same underlying data layer and tool set. We build the Seller Agent first.

Why We Are Not Starting With AI

An AI model is a reasoning engine. It cannot act without tools, and its tools are only as reliable as the code and data behind them.

If we connect an AI model to untested Python functions and an unstructured database, the agent will hallucinate, return incorrect stock levels, invent prices, and make decisions on corrupted data. Enterprises have already learned this the hard way — months of upfront planning, then the AI fails on day one because the data layer was never validated.

Our approach: build and test every tool the agent will use before the agent touches it. The AI is the last component we add, not the first.

How We Are Building It — The Delivery Method

We use **Scrum with 2-week sprints**. Every sprint delivers a working, tested slice of the system. Requirements are never fixed upfront. We treat every AI feature as an experiment with a measurable outcome.

The AI Delivery Loop

Every sprint runs this cycle:

Experiment → build the feature and write the test set
Evaluate → run against real cases, measure accuracy and latency
Improve → fix failures, refine prompts, add guardrails
Validate → human reviews edge cases and approves
Ship → stories that pass all criteria move to Done

Then the loop repeats.

The Measurable Criteria Rule

Every non-functional requirement must carry a specific, testable number. Vague requirements are rewritten before any story enters a sprint.

Not acceptable	Acceptable
The agent should be fast	Reply arrives in under 5 s at p95 across 20 test runs
No hallucinations	Zero invented stock values in 20 of 20 test cases
Offers should be fair	Counter never falls below the supplier floor price

Human-in-the-Loop

The agent pauses and routes to a human when:

- The buyer's request falls outside the negotiation policy
- The order value exceeds a supplier-defined threshold
- The agent detects a prompt injection attempt

This is a design decision, not a fallback.

The Squad

One cross-functional team. All six specialists share a single Definition of Done.

Role	Responsibility
DEV	Python agent tools and business logic
DATA	Product catalogue, pricing rules, test datasets
EVAL	Prompts, hallucination measurement, latency benchmarks
QA	Test cases, acceptance criteria enforcement
UX	Supplier-facing screens and escalation flows
SEC	Tenant isolation, input validation, audit logging

Definition of Done

A story is only Done when all of the following are true:

- All acceptance criteria pass, including every Given/When/Then scenario
- Code has been reviewed in a pull request by DEV, QA, and EVAL
- All non-functional numbers are hit on the full test set
- Failure cases are logged with input, expected output, actual output, and category
- The Product Owner has accepted the demo

Weeks 1–6 — Building the Foundation

Before any AI model is connected, we build and validate the complete tool layer the agent will rely on.

Week 1 — Agile Planning and Story Architecture

We take the business problem and turn it into a structured, sprint-ready backlog.

The squad defines two epics — one for the Seller Agent and one for the Buyer Agent. Each epic is broken into user stories in the standard format:

As a [role], I want [capability], so that [business value].

Every story is sized in story points using planning poker, given measurable acceptance criteria in Given/When/Then format, and tagged as either a Functional Requirement (what the system does) or a Non-Functional Requirement (how well it must do it, with a number attached).

Sprint 1 is planned. The squad commits Story 1.1 (check stock before quoting, 5 pts) and Story 2.2 (counter low offers automatically, 8 pts) against a team velocity of 20 pts.

Week 2 — Team Execution and Git Workflow

The squad runs Sprint 1 as a real cross-functional team.

Story 2.2 is broken into five engineering tasks, assigned to DEV, DATA, QA, and EVAL, with dependencies mapped before Day 1. The Daily Scrum surfaces a blocker on Day 5 — the DATA schema is late, which has cascaded to block both DEV and EVAL. The Scrum Master resolves it the same day.

Every code change goes through a pull request reviewed by at least two roles. For any task that touches the AI prompt or output logic, EVAL must be one of the reviewers.

The squad also establishes its failure log — a structured record of every test case the agent fails, with the input, expected output, actual output, and a category label. This log becomes the source of truth for prompt refinement in later sprints.

Week 3 — Python Tool Layer

With the sprint plan validated, we write the first real code: the Python functions the Seller Agent will call as tools.

`check_stock(sku)` queries the product catalogue and returns the available quantity for a given SKU. `counter_offer(listed_price, buyer_offer, floor_price, stock_level)` applies the supplier's negotiation policy and returns a safe counter price. Neither function contains any AI logic. They are deterministic, tested independently, and designed to be called by an agent tool wrapper.

Data moves between functions as Python dictionaries — the same structure that AI models use when communicating with external tools over JSON. This is intentional. The data shapes we define now become the schema the agent expects later.

Week 4 — Structured Codebase and File Handling

The individual functions from Week 3 are organised into a proper Python package.

A `ProductCatalog` class loads supplier inventory from a JSON file, exposes query methods, and handles file errors cleanly. A `PricingEngine` class encapsulates the negotiation rules, floor price logic, and MOQ (minimum order quantity) constraints. Both classes follow the same structure used by major AI frameworks — methods map directly to agent tool definitions.

The squad also writes the first version of the supplier data file — a structured JSON catalogue with product SKUs, listed prices, floor prices, stock levels, and MOQ values. This file is the Seller Agent's ground truth for every negotiation.

Week 5 — Persistent Storage

The JSON catalogue from Week 4 is sufficient for a single session. A production agent needs durable, queryable storage that survives restarts, supports concurrent access, and can be audited.

#	Do this	You should see
	The squad migrates to a PostgreSQL database. The product catalogue, pricing rules, and RFQ records are each given a dedicated table with appropriate constraints and indices. The Python functions from Week 3 are updated to query the database instead of the JSON file. The external interface — the function signatures and return shapes — does not change. The agent will not know the difference. The RFQ table stores every incoming buyer request with a timestamp, status, and the agent's response. This becomes the dataset for evaluating the agent's negotiation behaviour in Week 11. --- **Week 6 — Testing and Environment Safety** Every tool the agent will use is now written. Before the AI model is connected, we prove each tool works correctly under all conditions the agent is likely to create. The squad writes a `pytest` suite covering the happy path, boundary conditions, and failure cases for every function. `check_stock` is tested with valid SKUs, missing SKUs, and zero-stock SKUs. `counter_offer` is tested at the floor price boundary, above it, and below it — the below-floor case must never be returned under any input combination. Virtual environments lock the dependency versions. A `.env` file holds the database credentials and the future AI API key — neither ever appears in source code or in a commit. A `requirements.txt` captures the exact package versions so any team member can reproduce the environment in one command. The tool layer is now agent-ready.	
3.4	Save the file: **Ctrl + S** (Windows) or **Cmd + S** (Mac).	The dot disappears. The Source Control icon on the left gets a **1** badge, and the file name shows a green **U** (new file).

#	Do this	You should see

Stuck? No badge appears: the file is not saved, or the wrong folder is open. Check that the bottom-left corner shows `main`. **Prefer to edit instead:** click `README.md`, add a line, save. It will show an **M** (modified) instead of a U.

4 Commit and push

about 3 min

#	Do this	You should see
4.1	Click the **Source Control** icon in the left bar (the branch-shaped one), or press **Ctrl + Shift + G**.	A panel with a **Changes** list that includes `week2-demo.md`.
4.2	Move your mouse over the file row and click the **+** (Stage Changes).	The file moves from Changes to **Staged Changes**.
4.3	Click the **Message** box at the top of the panel and type `Add demo file`.	Your message appears in the box.
4.4	Click the blue **Commit** button. If VS Code asks whether to stage all changes and commit them directly, click **Yes**.	The lists empty. The blue button now says **Sync Changes** with a small up arrow and the number 1.
4.5	Click **Sync Changes**. If a box asks you to confirm that it will pull and push commits, click **OK**. If a sign-in window appears, sign in.	A short spinner, then the up arrow and number disappear. Your commit is on GitHub.
4.6	Open your repository page on GitHub (the original for Path A, your fork for Path B) and refresh the page (F5).	`week2-demo.md` is in the file list, with `Add demo file` next to it. On a fork you may also see "This branch is 1 commit ahead of testing03-03:main".

Stuck? 403 or "permission denied": you are on Path A without write access. Delete the local folder and repeat Step 2 using Path B. **"Remote has new changes":** click Sync Changes again. It pulls first, then pushes. **No Sync Changes button:** confirm the commit worked, meaning the Changes list is empty.



You are done when

- [ ] You cloned a repository to your computer.
- [ ] You created `week2-demo.md` (or edited `README.md`).
- [ ] You staged it, committed it with the message `Add demo file`, and synced.
- [ ] You can see the file and the commit message on github.com.

Plan B: the same steps in the terminal

If a button will not work, these commands do the same job. Open the terminal, move into your project folder with `cd`, and run them in order. Use your own fork address on Path B.

```
git clone https://github.com/testing03-03/ecomMart.gitgit
cd PostgreSQL_Ex01_UNESCO
git add week2-demo.md
git commit -m "Add demo file"
git push
```

How they match the buttons: the **+** button is `git add`, **Commit** is `git commit`, and **Sync Changes** includes `git push`.

Quick fixes

If you see	Do this
"Please tell me who you are" when committing	Git has no name or email. Run the two `git config --global` commands from step 0.5, then commit again.
`git` is not recognized (Windows)	Close all terminals and VS Code, reopen, and retry. If needed, restart the computer.

If you see	Do this
Source Control says it cannot find Git	Install Git (step 0.2) and restart VS Code.
Terminal says "not a git repository"	The terminal is in the wrong folder. Use **Terminal > New Terminal** inside VS Code with the project open.
The Sync Changes button is missing	The commit did not happen. Check that the Changes list is empty. If not, stage and commit first.
You want to practise again	Use a new file name, such as `week2-demo-2.md`. Or delete the file on GitHub (open it, three-dots menu, **Delete file**), delete your local folder, and clone again.

Sources and limits. Checked against the repository page and the VS Code documentation: Working with repositories and remotes and Source control overview. VS Code could not be run while writing this, so some button labels may differ slightly on your version. The "You should see" column is there to help you notice and adapt.